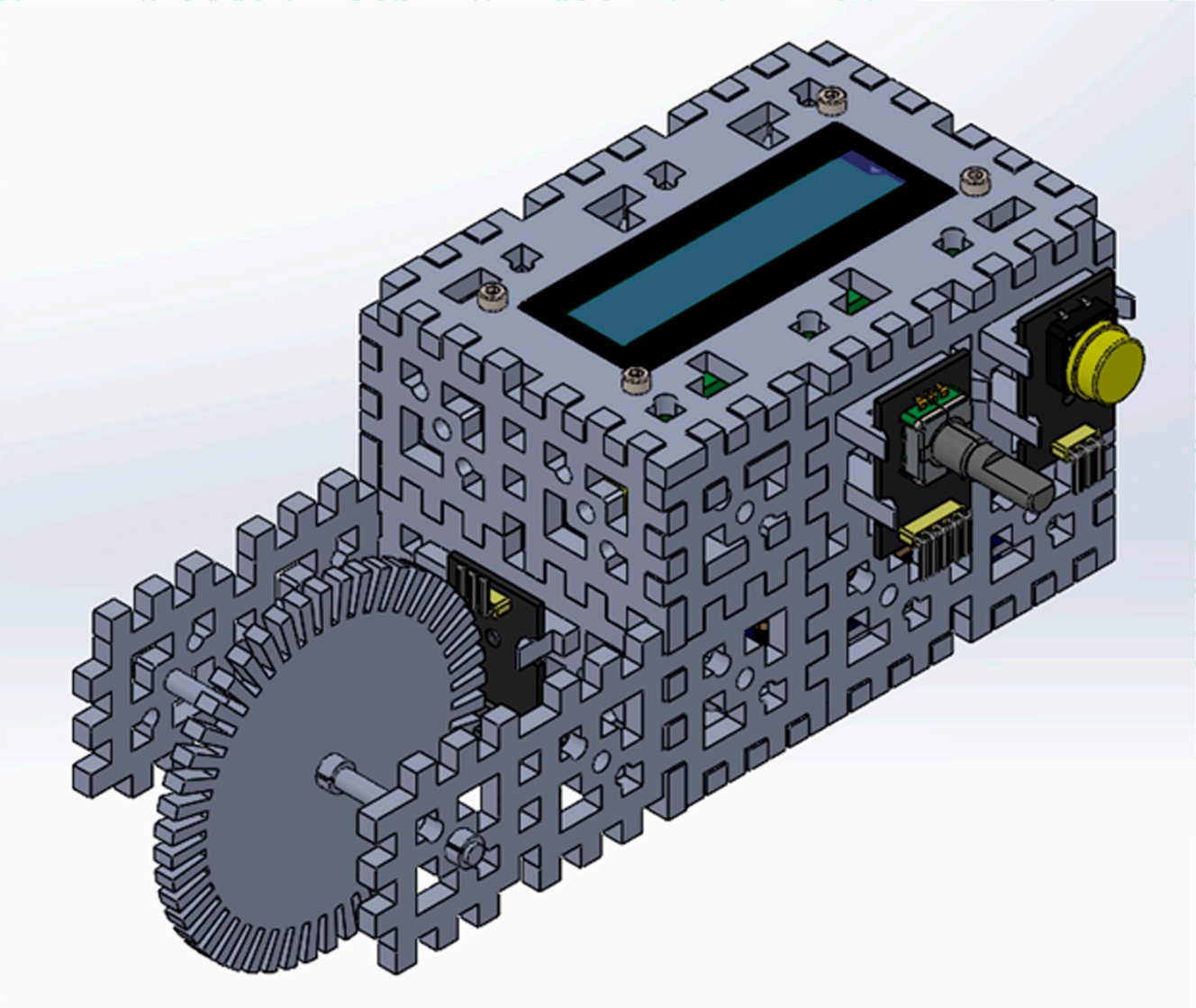
ПРОЕКТ №1
РОЗУМНА ЛІНІЙКА



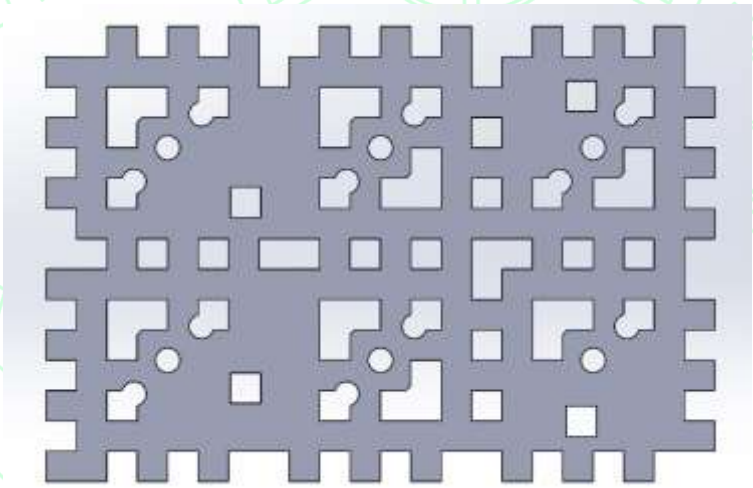
Мета:

- вивчити основні поняття про мікроконтролери;
- покращити навички складання електричних схем;
- покращити навички програмування мікроконтролерів;

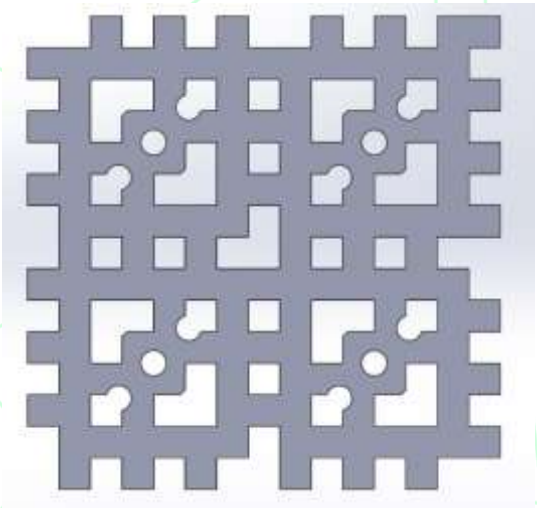
Необхідні електронні компоненти:

- мікроконтролер;
- плата розширення;
- дроти типу F-F;
- модуль фотопереривника;
- модуль енкодера;
- модуль потенціометра;
- ультразвуковий датчик відстані;
- модуль акселерометра MMA8452Q;
- модуль тактової кнопки;
- LCD-дисплей;

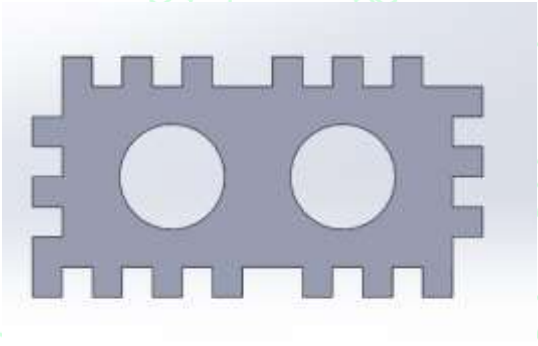
Необхідні конструктивні елементи:



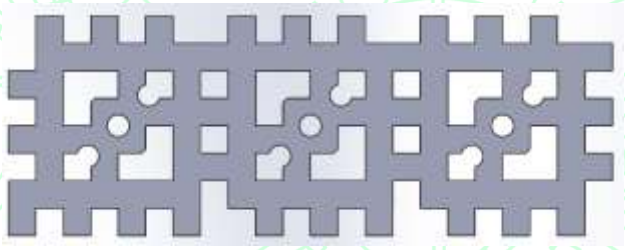
x1. CPlate2x3



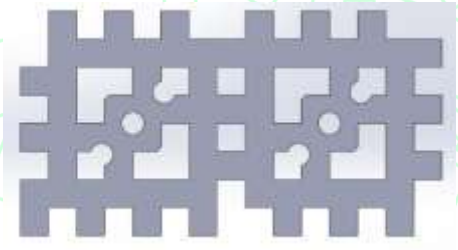
x2. Plate2x2



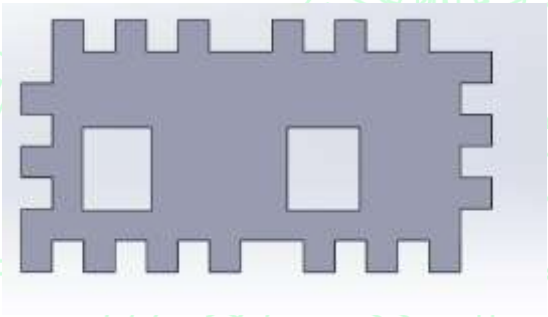
x1. SPlate 1x2



x2. Plate 1x3



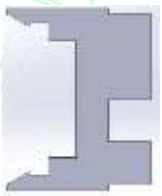
x2. Plate 1x2



x1. CPlate 1x2



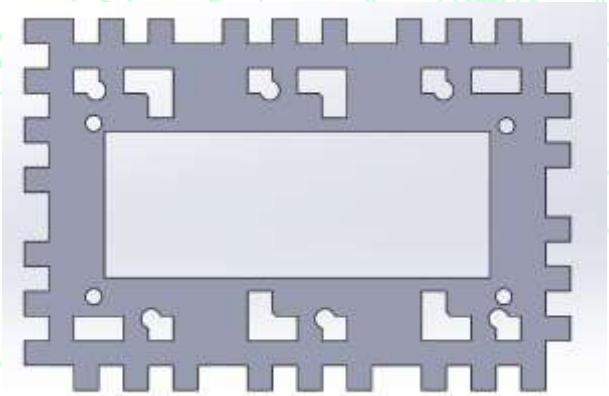
x4. Lfix



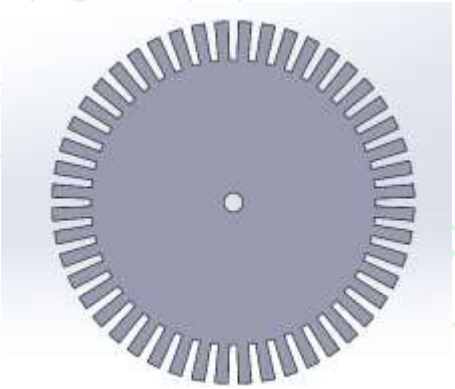
x4. Sfix



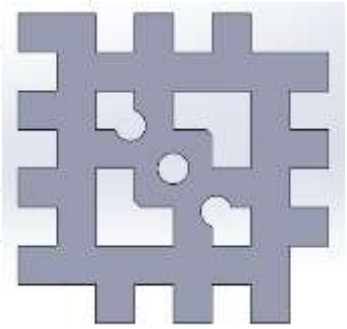
x4. AxS 4.7



x1. LCDPlate 1x3



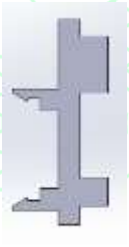
x1. WPInt70mm



x2. Plate 1x1



x1. LfixC



x1. SfixC



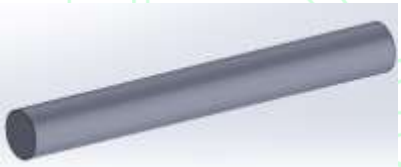
x4. Болт M2.5x10



x4. Гайка M2.5



x4. CPin



x1. Axis 3.7/75

Інформація

Люди завжди прагнули вимірювати різні кількісні величини - довжину, вагу, час. Незліченні винахідники, імена яких залишилися невідомими, поступово вдосконалювали конструкції вимірювальних приладів. У наш час створено прилади, що дозволяють виміряти навіть невидимі неозброєним оком предмети - наприклад частинки атомів.

Сучасна вимірювальна техніка для лінійних і кутових вимірів є результатом тривалого вдосконалення засобів вимірювальної техніки та вчення про вимірювання, тісно пов'язаних з розвитком землеробства, торгівлі, астрономії, мореплавання і будівництва. Але найбільший вплив на розвиток техніки лінійних вимірювань надали виробництво зброї, мануфактури, виробництво парових двигунів і т.д. Протягом багатьох років засоби і методи вимірювань удосконалювалися, але дуже повільно. Тільки в другій половині XIX століття, в зв'язку з швидким розвитком металообробної промисловості і розвитком науки і технологій почався прискорений прогрес вимірювальної техніки.

Історія вимірювальних інструментів починається, мабуть, саме з лінійки. Ще древні греки використовували такі дощечки з насічками, щоб вимірювати невеликі відстані. У тому вигляді, до якого ми звикли, лінійки з'явилися після Французької революції. Саме тоді для вимірювання були запропоновані металеві або дерев'яні дощечки, довжиною 1 метр з насічками по 1 сантиметру. Зараз лінійки також активно використовуються. Змінився матеріал, з якого вони робляться - замість металу і дерева часто використовується пластмаса. На них можуть відображатися різні розмірні шкали, пропонуватися не тільки прямі, але і хвилясті краї для проведення фігурних ліній і т.д..

Лінійка — найпростіший прилад для вимірювання довжини. Зазвичай лінійка має нанесені поділки, кратні тим одиницям вимірювання довжини (сантиметри, дюйми), які використовуються для вимірювання відстаней.

Ще 10-15 років тому навіть професійні будівельники користувалися звичайними схилами і рулетками. А зараз вже і на побутовому рівні нам доступні прилади, які автоматично розмічають площини з потрібним кутом нахилу, та проводять рівні лінії на стінах.

Рулетка є, напевно, в арсеналі всіх домашніх майстрів і професійних будівельників. Вона відмінно замінює лінійку, якщо потрібно виміряти відстань більше 1 метра. Перші прототипи рулеток з'явилися в 16 столітті в Китаї. Вони використовувалися для вимірювання відстаней на відкритій місцевості. Це був рулон тканини, на якому відзначалися міри довжини. Скочувався він, звичайно, не автоматично, а вручну. Але все-таки це було набагато зручніше, ніж ходити по ділянці зі звичайною лінійкою або аршином.

Поступово рулетки розвинулися в більш компактні і зручні прилади. Тканина в їх корпусі замінила міцна стрічка з металу. Така стрічка має ребро жорсткості, що проходить в центрі по всій довжині, завдяки чому полотно не згинається і їм зручно користуватися.

Незважаючи на те, що користуватися такими рулетками зручно, по суті, вони все одно недалеко пішли від свого китайського прототипу, винайденого кілька століть назад. Нещодавно на ринку з'явилися принципово нові сучасні вимірювальні інструменти, які роблять заміри не за допомогою полотна, а за допомогою лазерного променя. Їх головна перевага - це простота у використанні. Відстань від точки до точки вимірюється з мінімальною похибкою. При цьому навіть більшу довжину може виміряти одна людина без допомоги.

Для вимірювання кутів та рівня використовувалися так звані схили. Це найпростіший прилад, що складається з дерев'яного прямокутного трикутника і прикріпленого до його вершини грузка на мотузочці. З його допомогою проводилася спочатку пряма вертикальна лінія, а від неї вже відлічувалися потрібні по горизонталі кути. Але в 1666 році з'явився перший рівень з прозорою колбою - прототип тих вимірювальних приладів, які зараз знайомі кожному будівельнику. Поступово розвиваючись, рівні ставали більш складними. На них з'явилася додаткова колба, розташована перпендикулярно до основної.

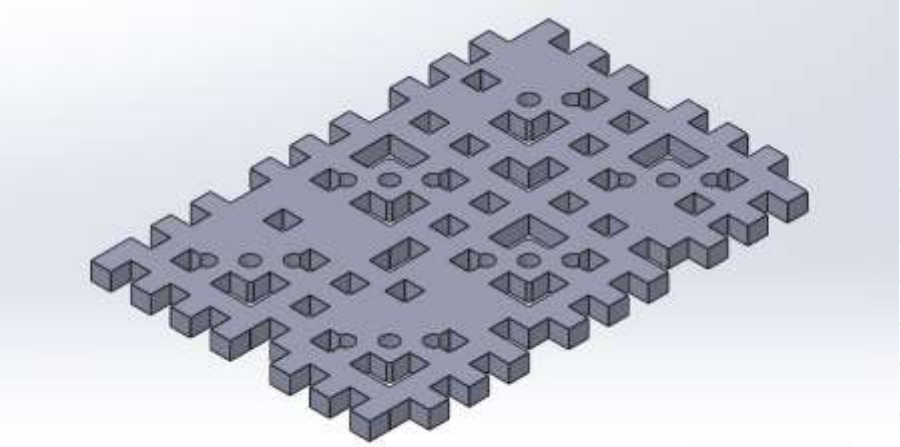
Доступні зараз і сучасні лазерні моделі рівнів. Вони дозволяють провести рівну лінію по стінах, проводячи вертикалі і горизонталі в трьох вимірах. Лінії просто проектуються лазерами. Їм можна задати необхідний кут нахилу, що особливо важливо при втіленні в життя складних дизайнерських проектів. Професійні моделі таких рівнів оснащуються системою автоматичного вирівнювання.

В ході виконання даного проекту, ви створите власний розумний вимірювальний пристрій, використовуючи різні датчики. Наша розумна лінійка буде вміти вимірювати довжину на площині за допомогою диска с отворами та оптичного датчика, відстань до об’єкта в просторі за допомогою ультразвукового датчика відстані, рівень та кут нахилу за допомогою акселерометра.

Принцип роботи вимірювального пристрою дуже простий: за допомогою фізичного руху, пристрій одразу дозволяє виміряти відстань на площині, відстань до об’єкта, рівень та кут нахилу самого пристрою; всі дані збираються та виводяться на LCD-дисплей. Все це відбувається за допомогою 3-х датчиків:

Збірка моделі

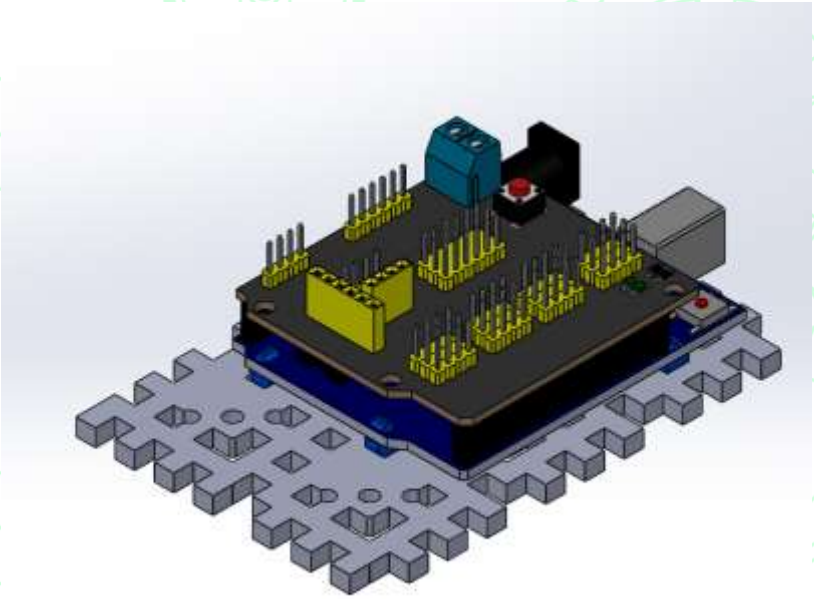
1. Підготуйте необхідні конструктивні елементи. Основою моделі буде пластина для кріплення мікроконтролера.



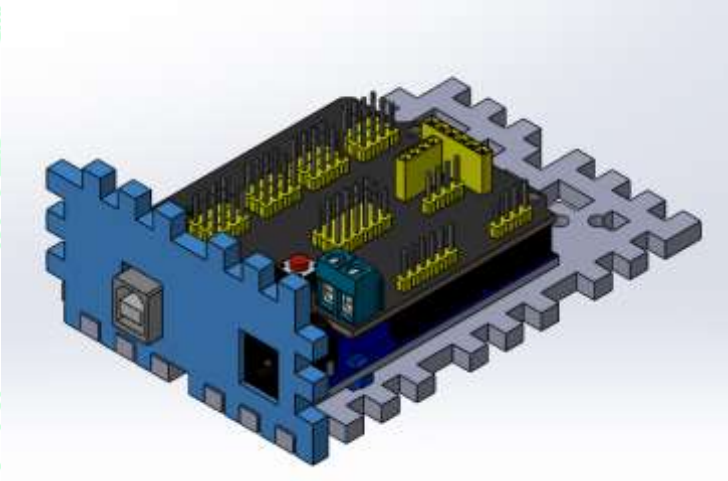
2. Додайте піни для кріплення мікроконтролера до пластини.



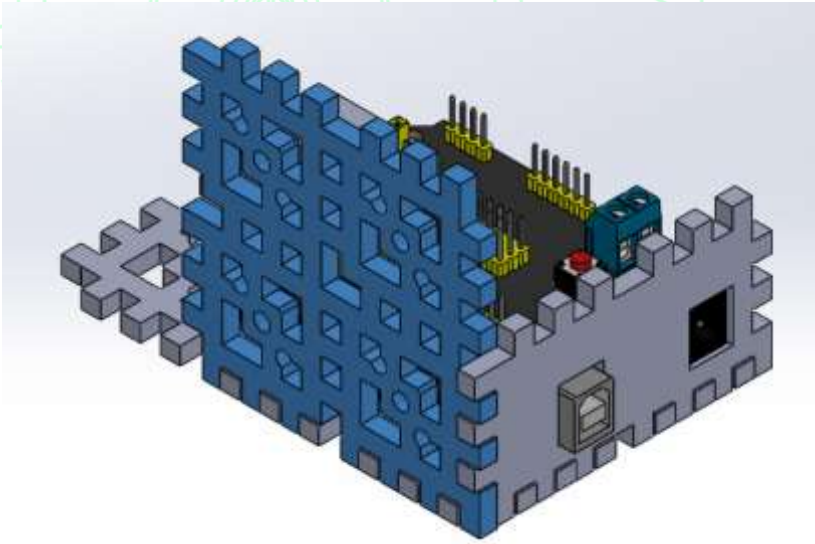
3. Додайте контролер із встановленою на нього платою розширення.



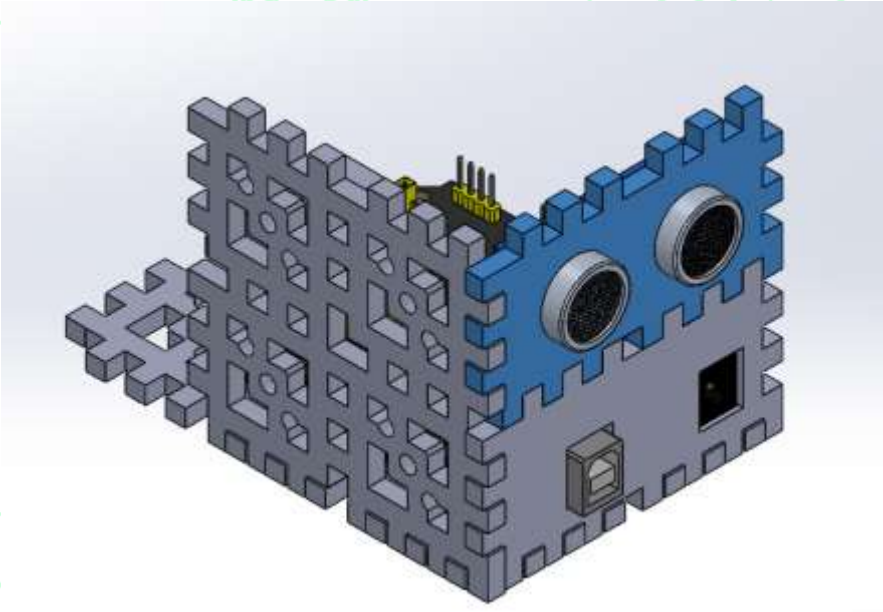
4. Додайте пластину 1x2 з вирізами під мікроконтролер.



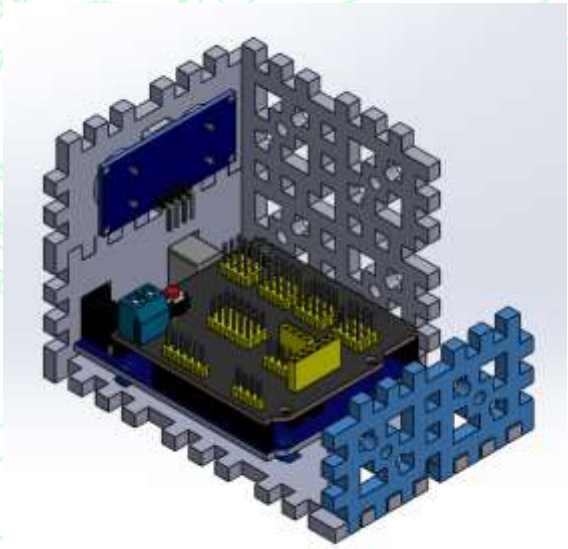
5. Додайте пластину 2x2 та зафіксуйте як показано на рисунку, для формування загальної конструкції корпусу.



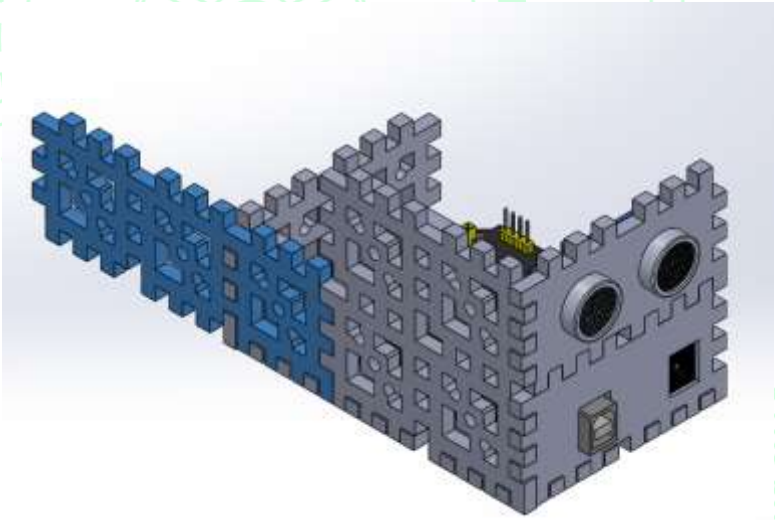
6. Додайте пластину 1x2 з вирізами під ультразвуковий датчик відстані разом з датчиком та зафіксуйте як показано на рисунку.



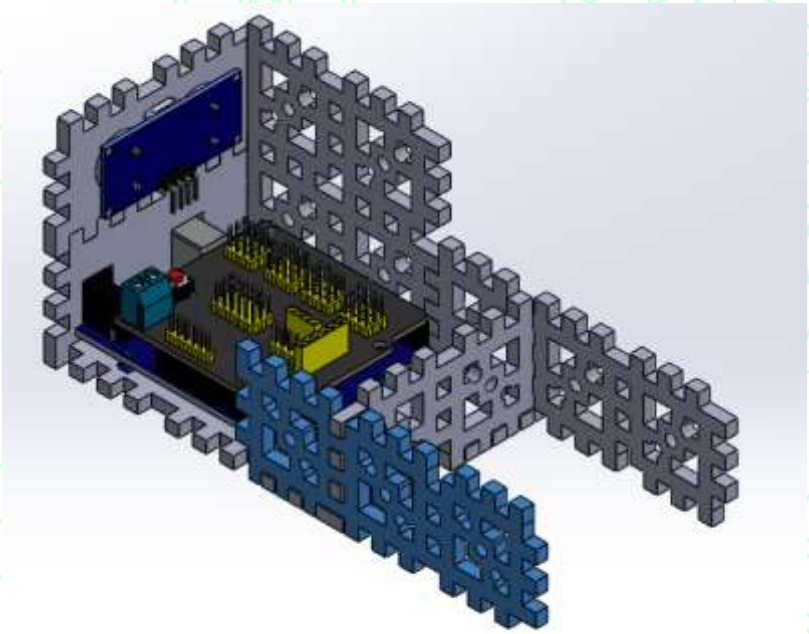
7. Додайте пластину 1x2 та зафіксуйте з протилежної частини моделі.



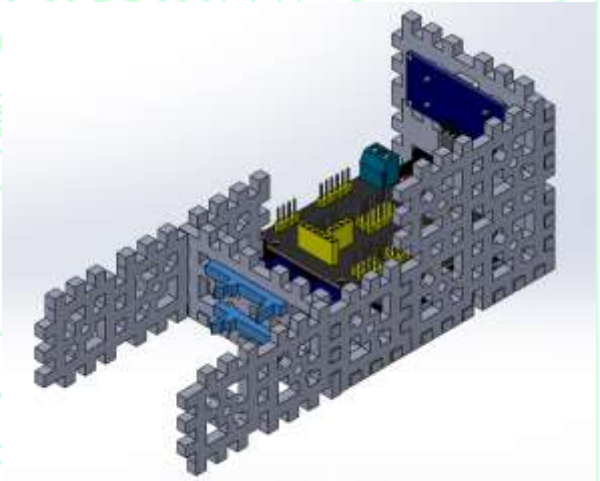
8. Додайте пластину 1x3 як показано на рисунку.



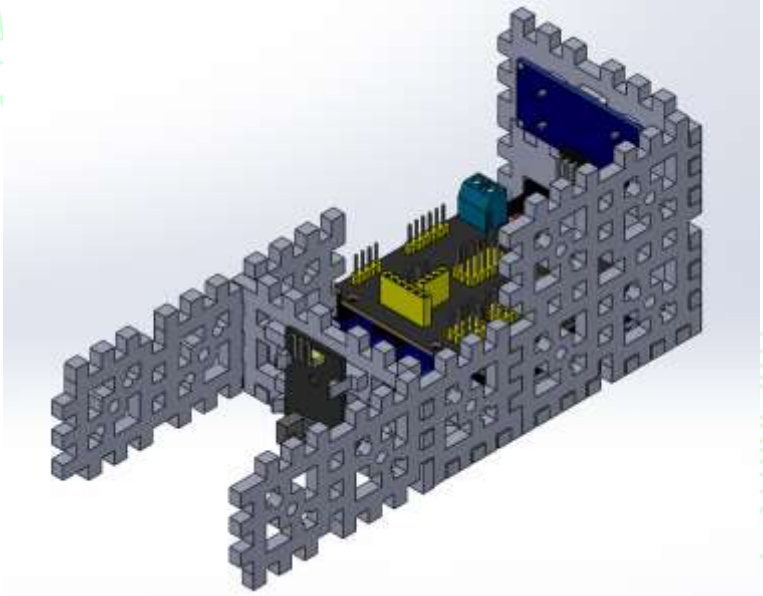
9. Додайте таку само пластину з іншого боку, це буде основою для кріплення диску з отворами.



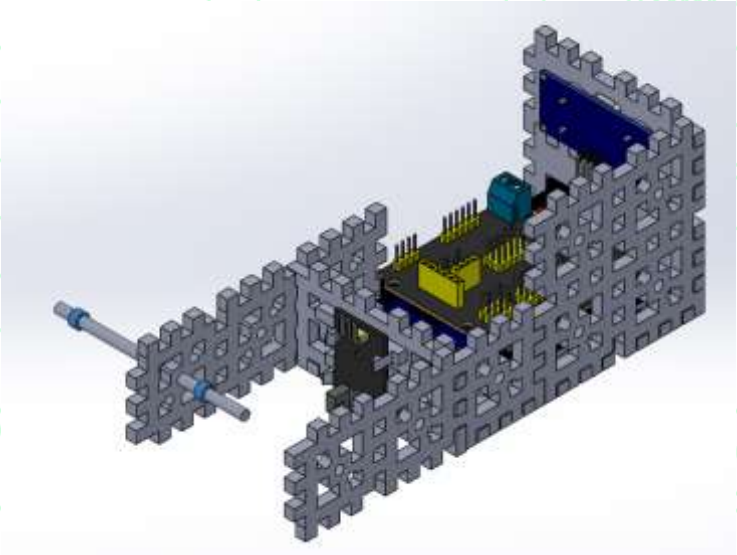
10. Додайте кріпленні для фіксації оптичного датчика як показано на рисунку.
Примітка. В цьому випадку вам необхідно використати кріплення для кріплення датчиків по центу. Такі кріплення ширші ніж звичайні.



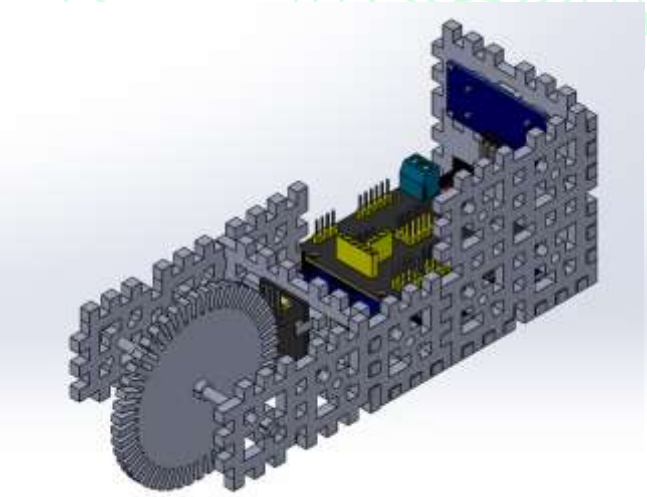
11. Побудуємо механізм для вимірювання довжини на площині. Додайте модуль оптичного датчика – фотопереривника, та зафіксуйте як показано на рисунку.

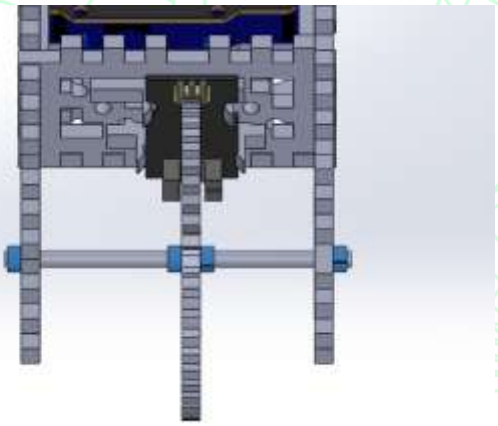


12. Вставте вісь 3.7/75mm наполовину в крайні центральні отвори пластини 1x3. Відразу надіньте на вісь два кінцеві фіксатори як показано на рисунку.

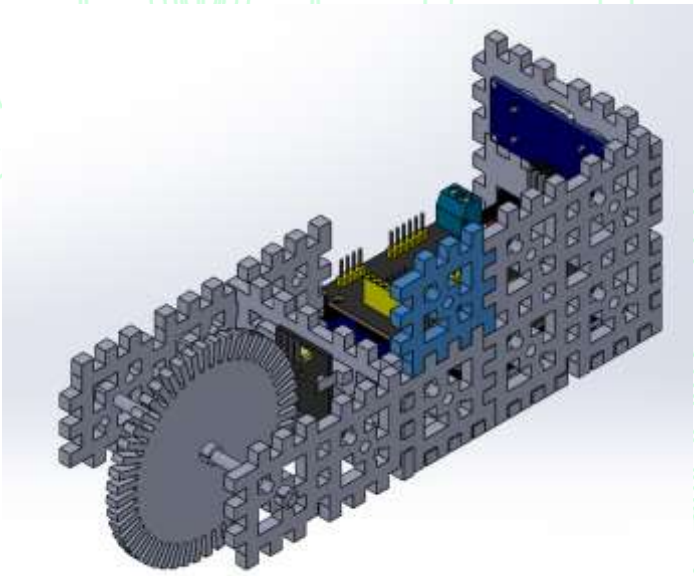


13. Додайте диск з отворами та ще два стопорні кільця. У вас повинно вийти як показано на рисунку.
Примітка. Диск повинен знаходитись рівно у вирізі модуля фотопереривника.

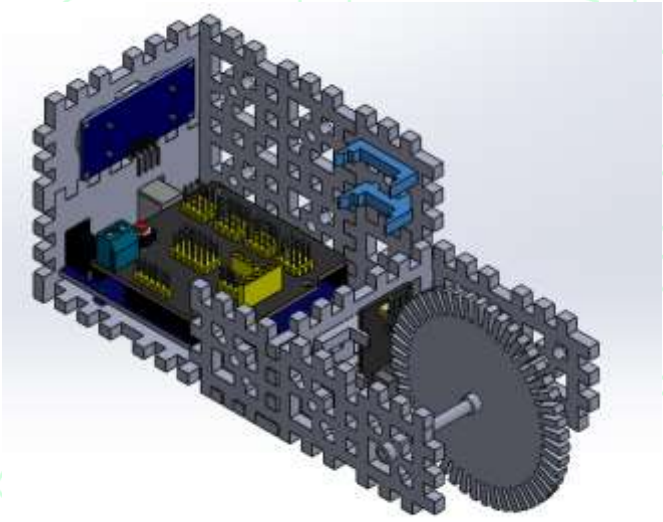




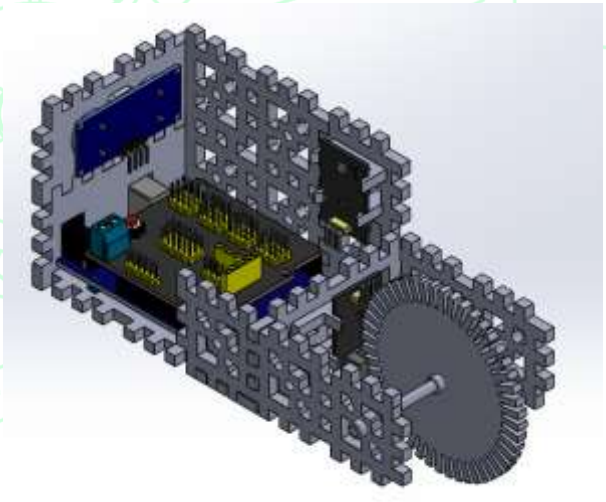
14. Додайте пластину 1x1, та зафіксуйте як показано на рисунку.



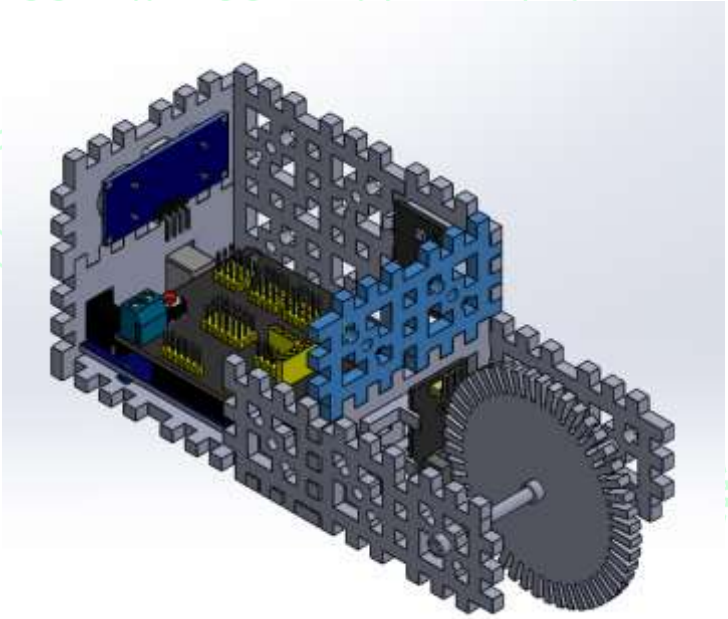
15. Додайте кріплення для модуля акселерометра.



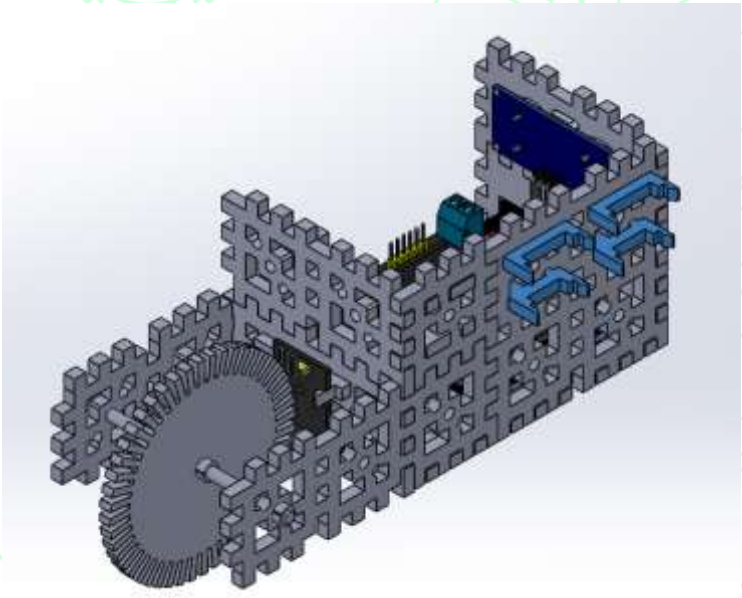
16. Додайте модуль акселерометра.



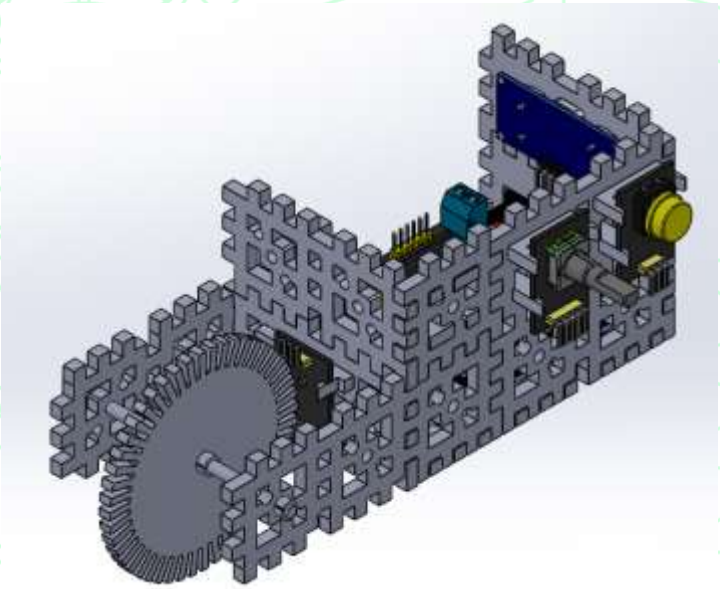
17. Додайте пластину 1x2 та зафіксуйте як показано на рисунку.



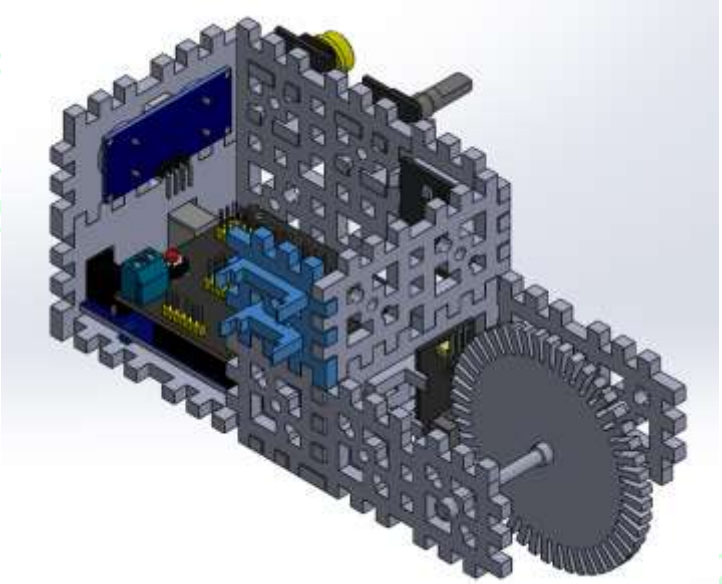
18. Додайте кріплення для модуля кнопки, та модуля енкодера.



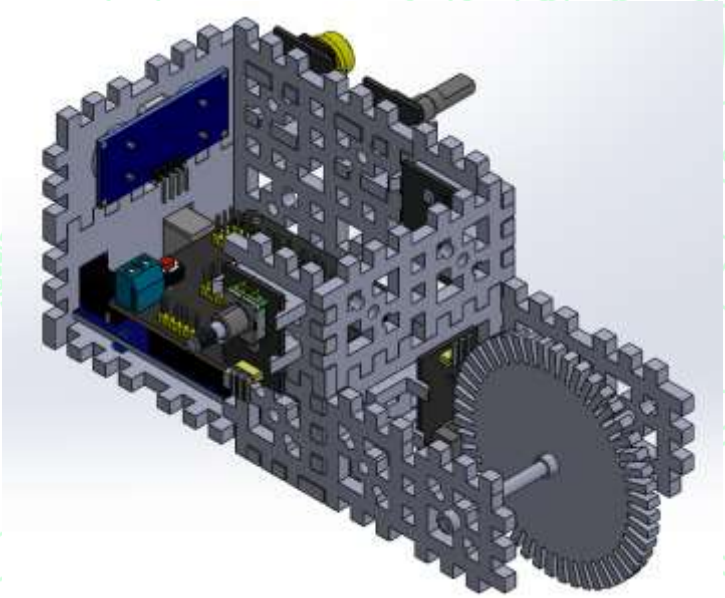
19. Додайте модуль кнопки, та модуль поворотного енкодера.



20. Додайте пластину 1x1, та кріплення для датчика з іншої сторони.

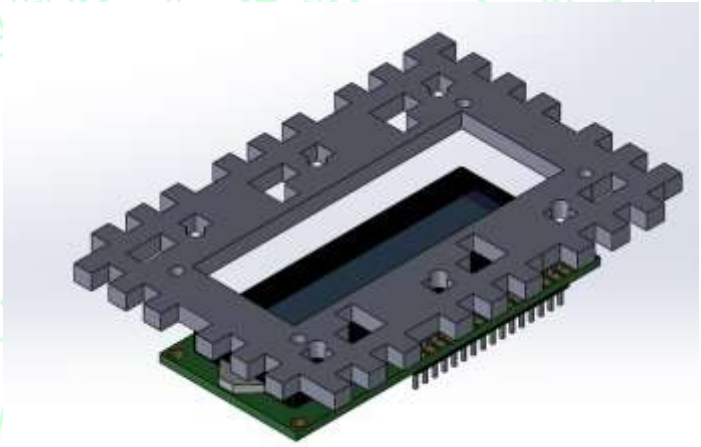


21. Зафіксуйте потенціометр як показано на рисунку.

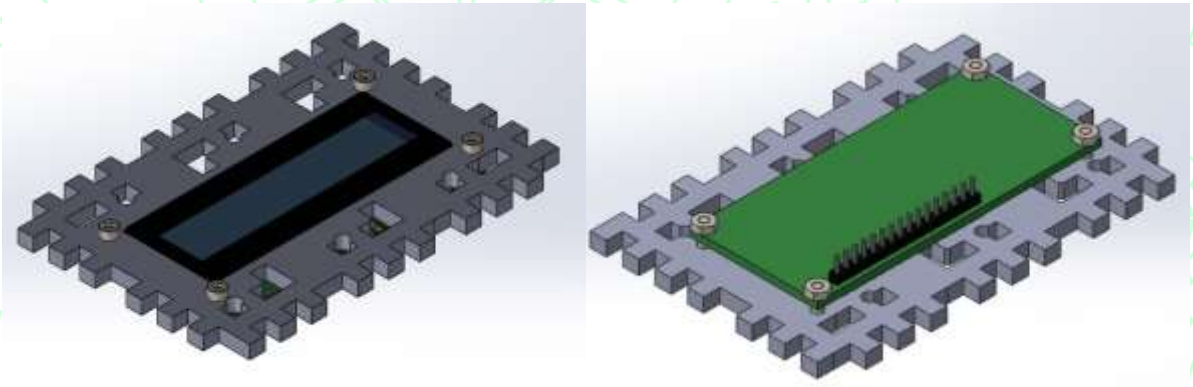


22. Зафіксуємо LCD дисплей на пластині. Для цього нам знадобиться пластина для кріплення LCD дисплею, сам дисплей, чотири болти M2.5 x10мм., та чотири гайки M2.5.

Вставте LCD дисплей в пластину як показано на малюнку.



23. Зафіксуйте LCD дисплей за допомогою чотирьох болтів та гайок.



24. Наступним кроком необхідно з’єднати всі електронні компоненти відповідно до схеми. Для зручності сприйняття схема розділена на дві частини, але по закінченню у вас повинні бути підключені всі компоненти.

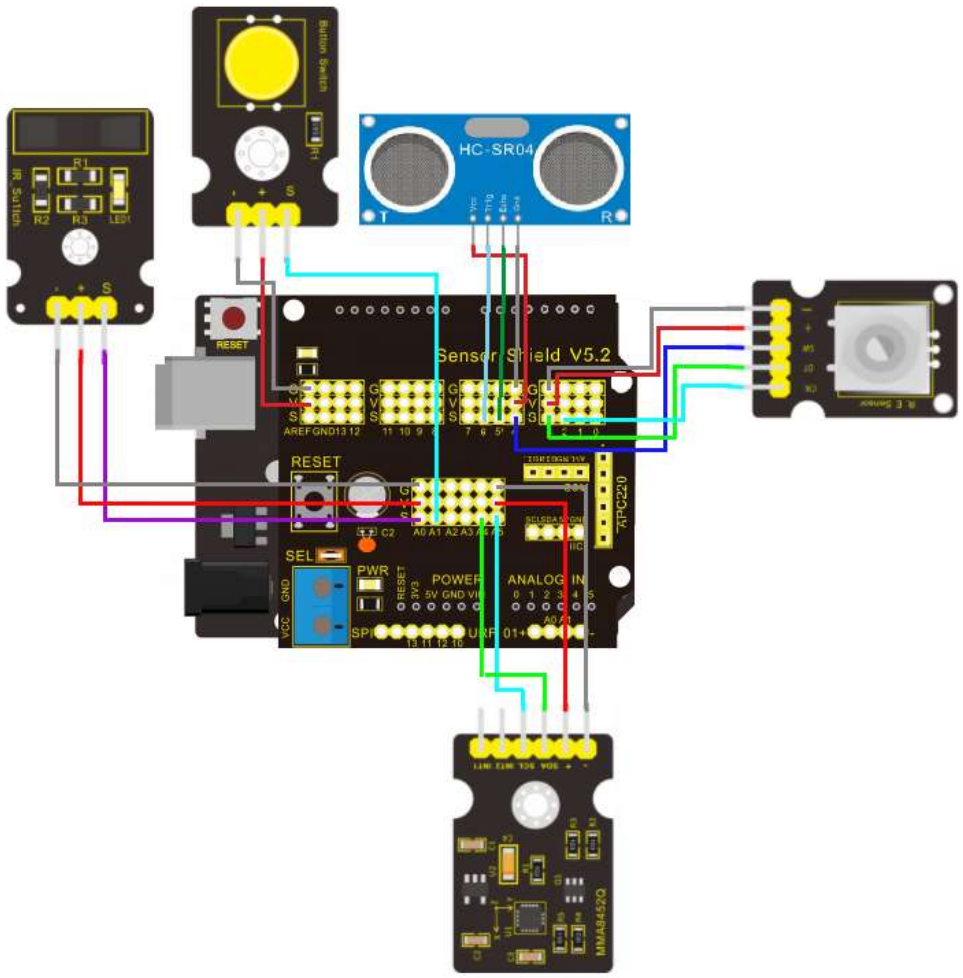


Схема підключення частина 1.

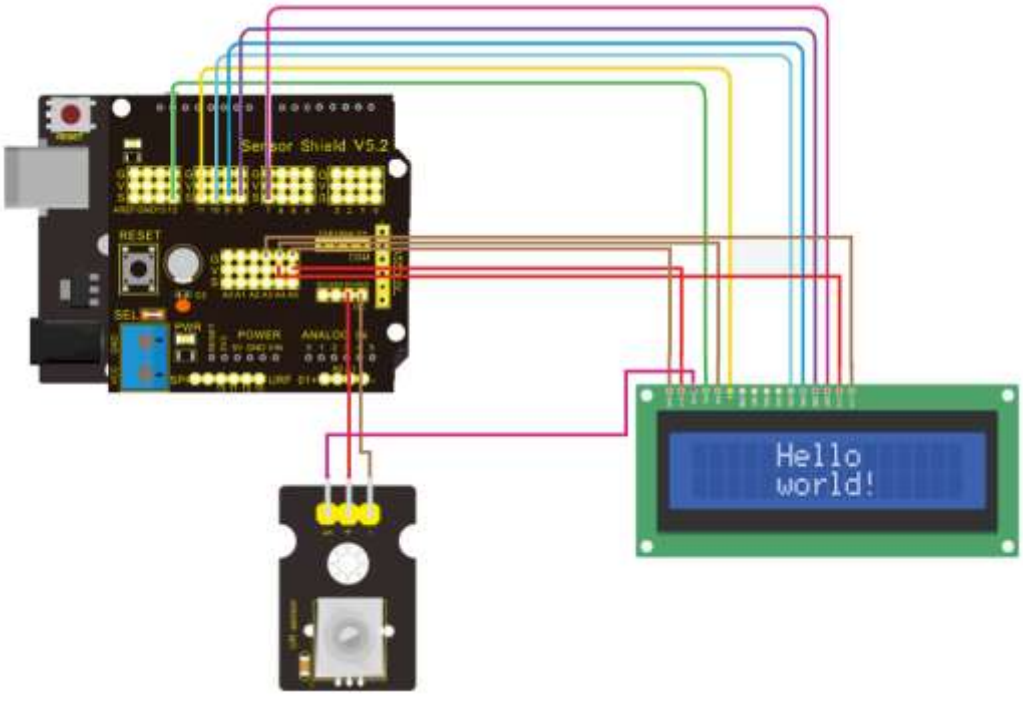
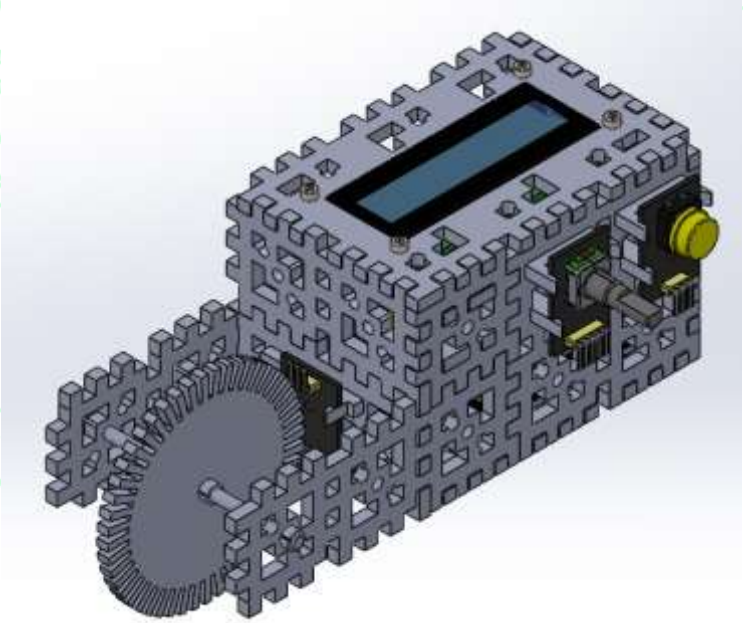
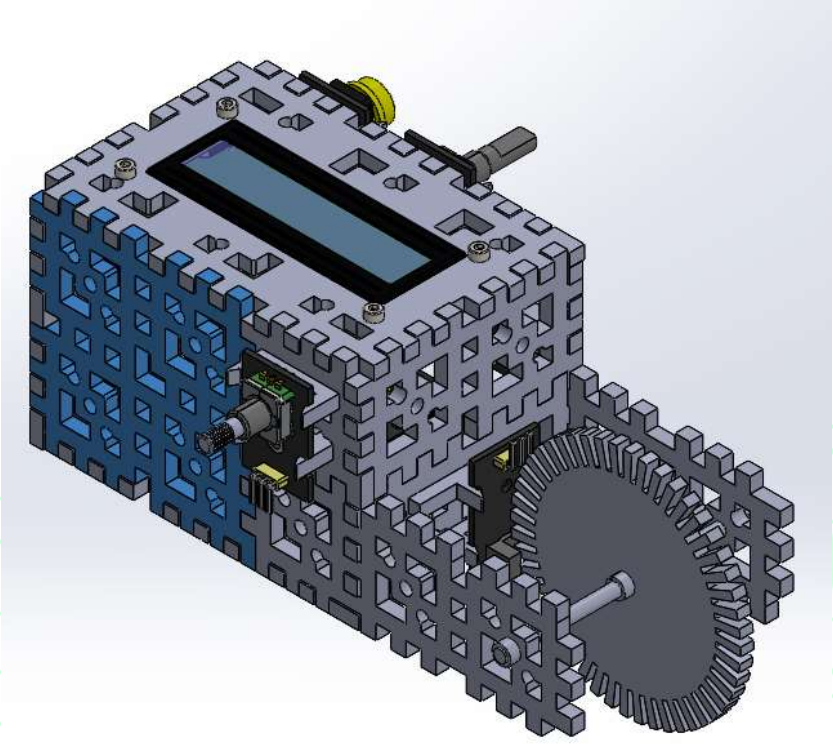


Схема підключення частина 2

25. Зафіксуйте LCD дисплей на моделі, перед фіксацією уважно перевірте необхідні електронні підключення.



26. Для завершення конструкції зафіксуйте пластину 2x2 як показано на рисунку.



27. Після збирання моделі необхідно запрограмувати наш розумний вимірювальний пристрій.

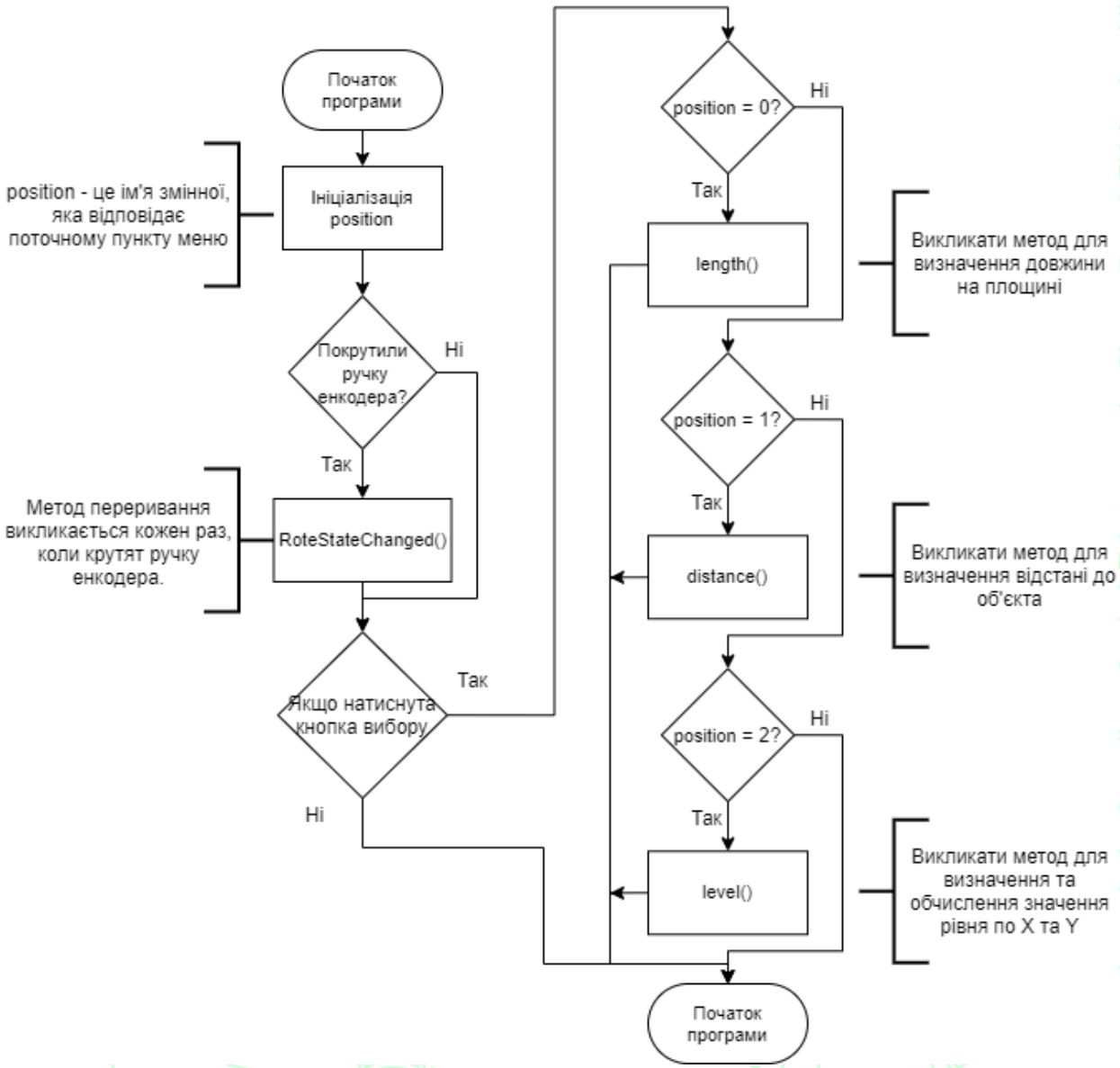
Наш вимірювальний пристрій буде мати декілька режимів роботи:

1. Вимірювання довжини на площині;
2. Вимірювання відстані до об’єкта в просторі;
3. Положення в просторі по осям X та Y;

Отже, для перемикання режимів роботи нам необхідно реалізувати меню з можливістю вибору. Для відображення меню ми використовуємо LCD дисплей а в якості елемента контролю для вибору необхідного режиму, будемо використовувати інкрементальний енкодер.

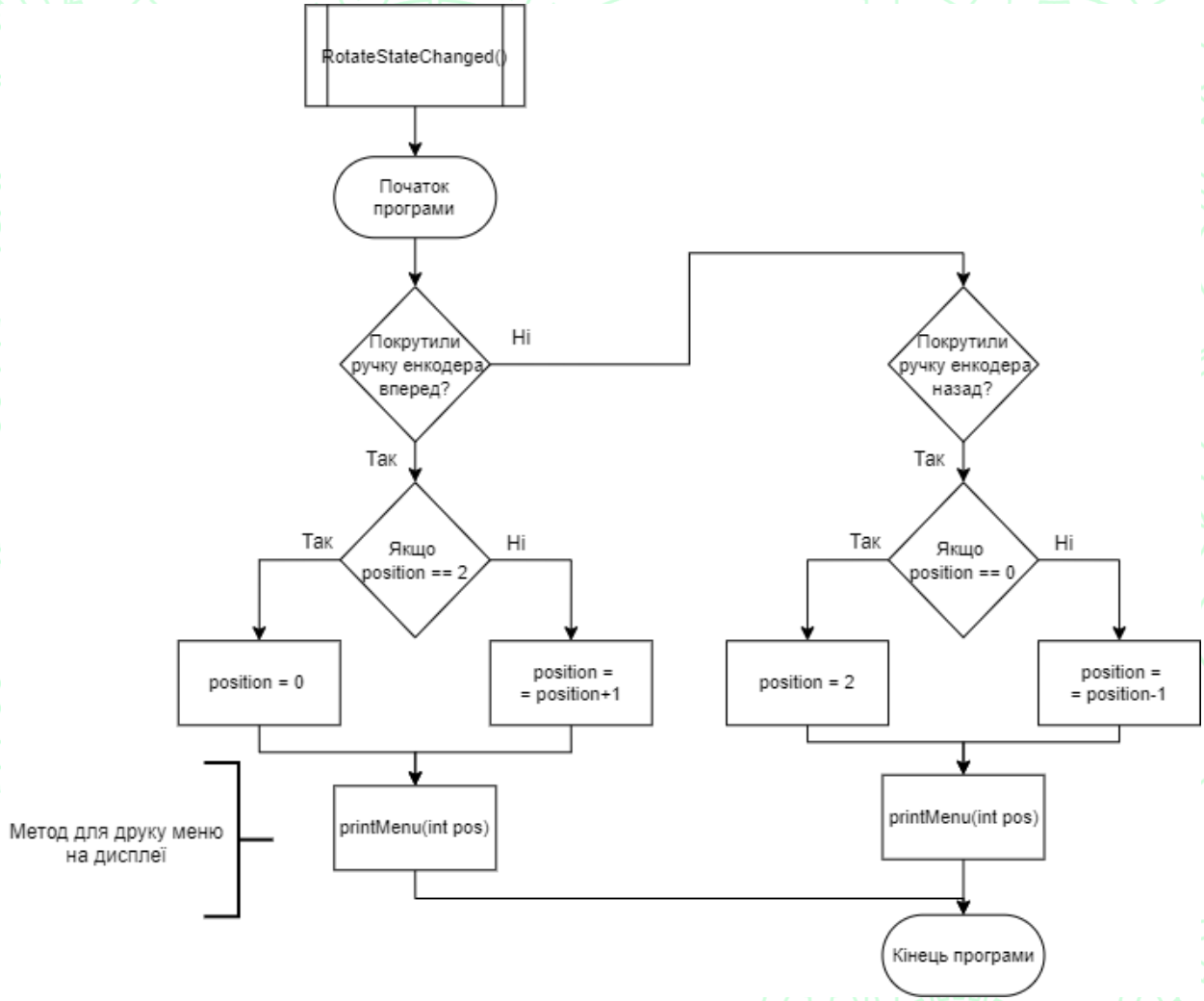
Примітка. Для того щоб згадати принцип роботи енкодера зверніться до лабораторної роботи №5.

Загальний принцип роботи пристрою зображено на блок-схемі.



Як видно з блок-схеми, при повороті ручки енкодера викликається метод який буде змінювати змінну «**position**» в залежності від напряму обертання, а при натисканні кнопки енкодера, буде викликатись метод у відповідності до значення змінної.

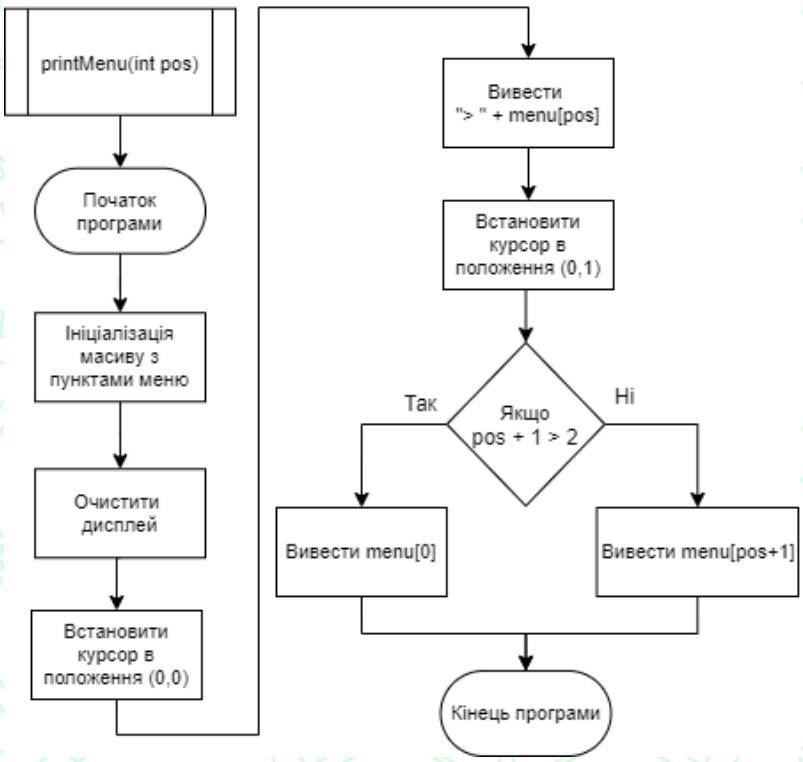
Давайте розглянемо блок-схему методу **RotateStateChanged()**, щоб зрозуміти яким чином змінюється змінна всередині нього, та як від цього залежить вигляд меню яке друкується на дисплеї.



Виклик методу **RotateStateChanged()** зав'язаний на апаратному перериванні, це означає що він буде викликатись у випадку якщо на піні який ми назначили на переривання буде змінюватись сигнал, а оскільки пін підключено до енкодера, то метод буде викликатись тільки у випадку якщо покрутили ручку.

Грунтуючись на принципі роботи енкодера, ми знаємо що в залежності від напряму обертання на піні **dat** енкодера буде або логічна одиниця, або логічний нуль, таким чином ми можемо і дізнатись напрям обертання та змінити значення змінної яка відповідає за порядок відображення меню. Оскільки у нас в меню тільки чотири позиції, необхідно слідкувати щоб змінна «**position**» знаходилась в діапазоні від 0 до 2. Отож якщо змінна набула крайнього значення, ми присвоюємо їй значення протилежне, якщо крайнього значення ще не набула то збільшуємо або зменшуємо на одиницю, після чого викликаємо метод **printMenu()** для відображення меню на дисплеї.

Примітка. Для того щоб згадати як працюють переривання зверніться до лабораторної роботи №5.



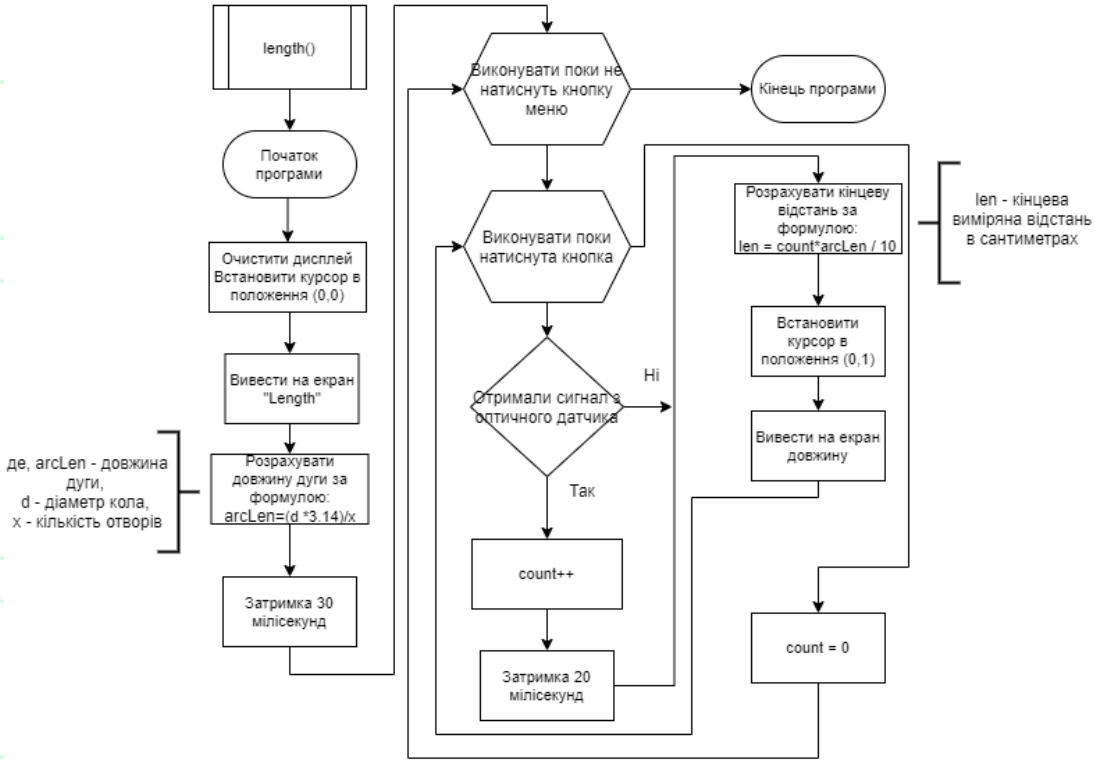
Блок-схема методу для відображення меню на дисплеї виглядає наступним чином.

Для відображення меню спочатку ми створюємо масив з чотирьох елементів типу **String** для зберігання всіх пунктів меню по черзі.

Метод приймає один аргумент типу **int** який вказує на поточну позицію елемента меню який обирає користувач поворотом ручки енкодера.

Дисплей має тільки два рядки для виводу, отож ми можемо вивести тільки два пункти одночасно. Пункт меню на який вказує змінна position буде виводитись першим, для зручності користувача ми додаємо знак стрілочки «>» перед пунктом, для того, щоб користувачу було легше зрозуміти який пункт меню зараз активний. В якості другого рядка ми виводимо наступний елемент масиву, якщо він існує, якщо ж змінна вказує на останній елемент, то ми виводимо перший елемент меню в якості другого пункту.

Давайте розглянемо блок-схеми для вимірювання всіх величин, щоб краще розібратись в принципі роботи нашого пристрою. Розглянемо метод для вимірювання довжини на площині.



Для вимірювання довжини на площині ми використовуємо диск з отворами та фотопереривник. Датчик має два елементи, один з який відправляє світловий потік, а інший сприймає. Якщо між передатчиком та прийомником знаходиться об’єкт, відповідно сигнал не буде отриманий. Таким чином, коли ми будемо проводити нашим пристроєм по площині, то буде крутитись колесо тим само то пропускаючи то зупиняючи світловий потік. Знаючи довжину кола ми можемо знайти відстань яку воно пройшло.

Для того щоб знайти довжину кола скористаємось формулою $(d * 3.14)$

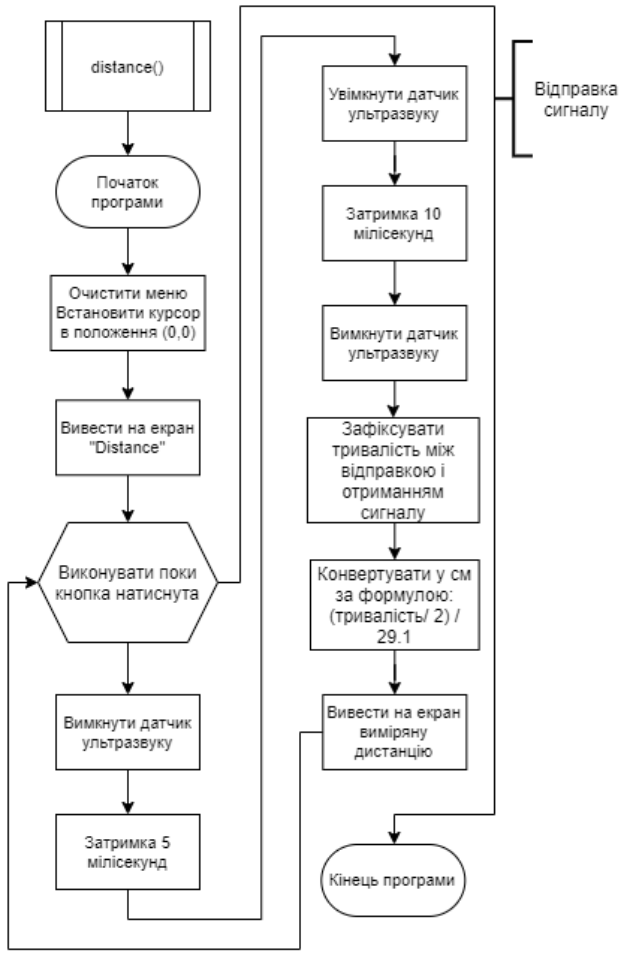
Для того щоб отримати більш точні результати на диску знаходиться 48 отворів, відповідно коло ділиться на 48 рівних дуг, для того щоб знайти довжину дуги, необхідно поділити довжину кола на кількість дуг.

Для того щоб порахувати всі спрацювання датчика у нас є змінна «**count**» та додаткова кнопка, для того щоб рахувати оберти тільки коли вона натиснута, такий підхід допоможе в вимірюваннях тим що оберти без натискання кнопки не будуть зараховані.

Отже порахувавши кількість спрацювань датчиків, ми знаємо скільки обертів здійснило колесо, помноживши довжину дуги на кількість обертів ми знайдемо відстань в мм, та поділимо на 10 щоб отримати відстань в сантиметрах. Після чого просто виводимо результат на дисплей.

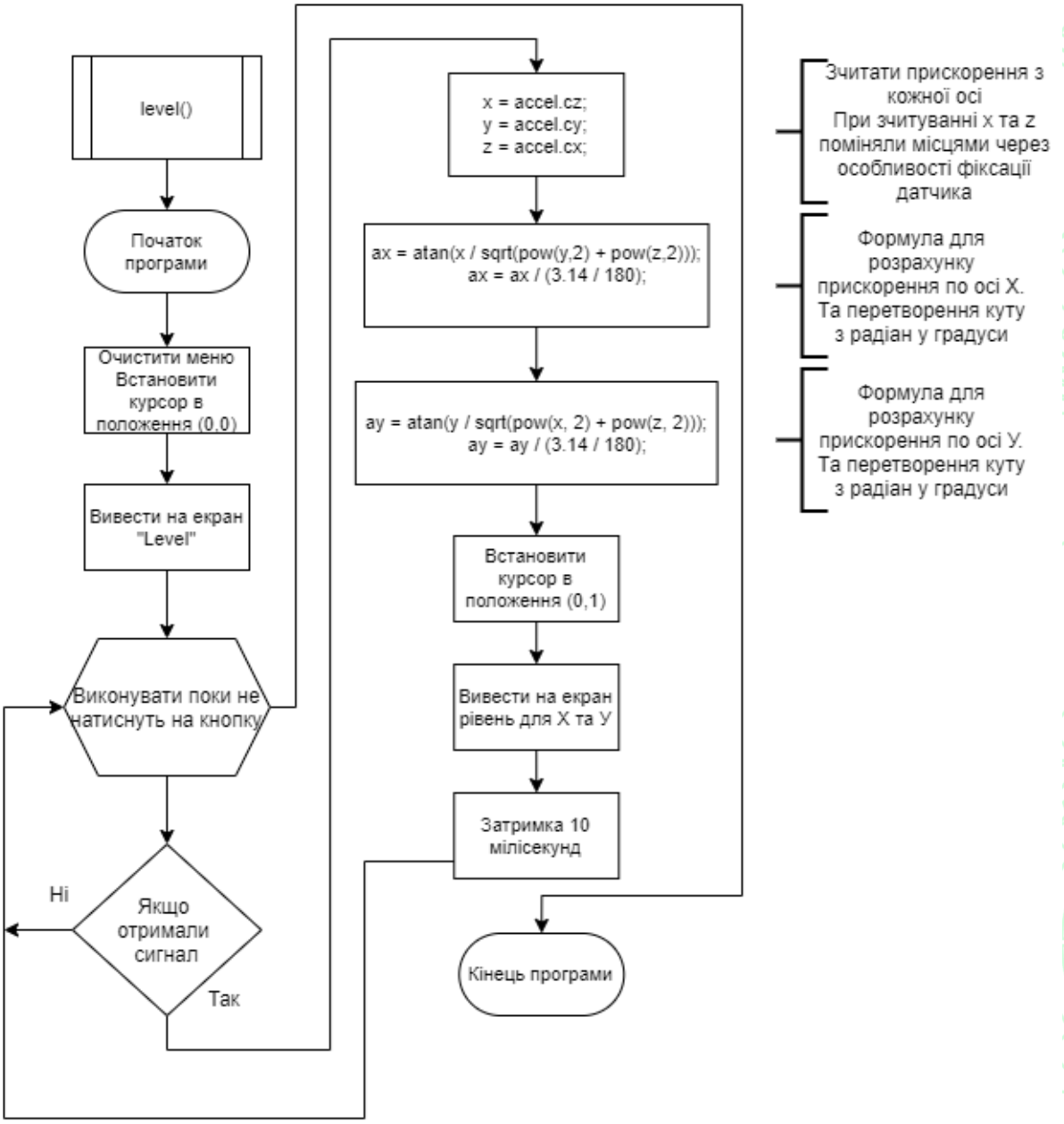
Наступний метод **distance()** буде використовуватись для визначення відстані до об’єкта за допомогою ультразвукового датчика відстані.

Примітка. Принцип роботи ультразвукового датчика відстані ви можете згадати з лабораторної роботи №10 з базового комплекту.



Отже ми просто відправляємо сигнал з датчику та фіксуємо час між відправкою та отриманням. Знаючи швидкість з якою поширюється ультразвук ми знаходим відстань до об’єкта та виводимо отримане значення на дисплей.

Наступним кроком дослідимо принцип роботи методу для визначення положення пристрою в просторі або нахилу поверхні на якій він знаходиться.



Суть роботи методу визначення рівня, базується на властивостях акселерометру. Акселерометр це пристрій що вимірює прискорення по кільком осям. Якщо він знаходиться в стані спокою, він показує прискорення в залежності від сили земного тяжіння. Земне тяжіння є постійним, і в залежності від орієнтації акселерометра в просторі, воно буде по різному діяти на різні осі. Наприклад якщо акселерометр розташовано горизонтально то прискорення буде тільки по осі Z а по осям X та Y воно буде дорівнювати нулю. Зчитавши прискорення по кожній з осей ми можемо знайти положення акселерометру в просторі скориставшись формулами тригонометрії.

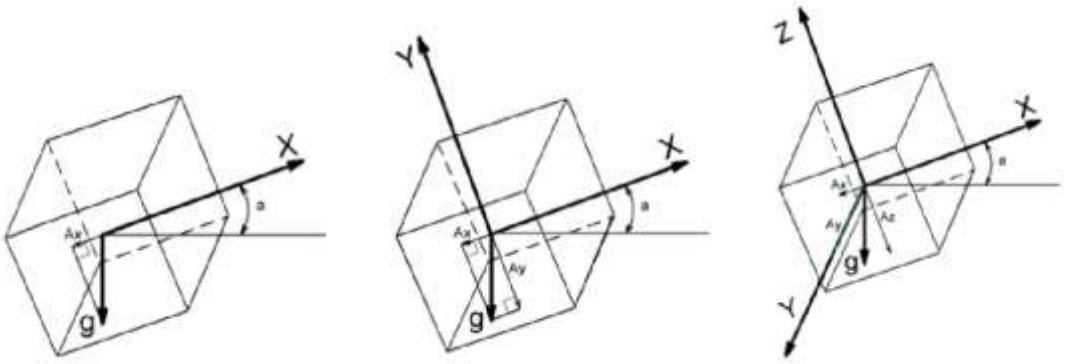


Рисунок 1. Положення акселерометра відносно сили гравітації: зліва – для однієї осі, по середині – для двох, справа – для трьох

Величина проекції залежить від кута нахилу по синусоїдальному закону $Ax = g\sin(a)$, де Ax проекція вектора на вісь, g – прискорення вільного падіння, a – кут між віссю акселерометра і горизонтом.

Відповідно для обчислення кута нахилу достатньо змінити формулу наступним чином: $\arcsin(\frac{Ax}{g})$. Але таким чином для визначення куту використовується лише одна вісь, що є недостатнім через те що при наближенні кута до 90 градусів чутливість акселерометра падає, відповідно краще буде визначати кут для кожної осі, базуючись на показання всіх трьох осей. Отже формула для визначення кута нахилу набуде наступного вигляду:

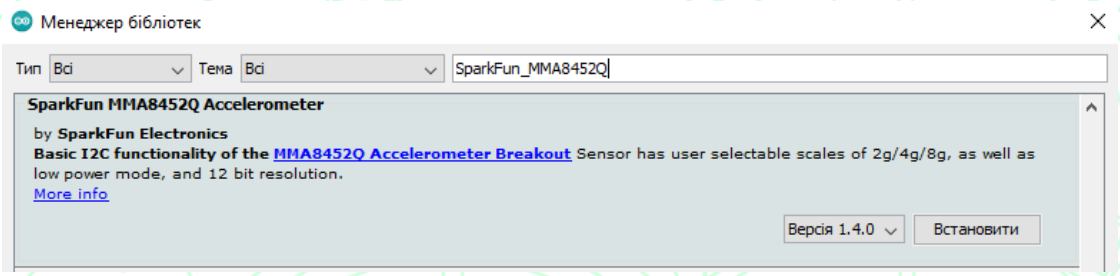
$$a = \arctan(\frac{Ax}{\sqrt{Ay^2 + Az^2}})$$

де Az – проекція вектора g на вісь Z.

Таким чином ми отримуємо кут нахилу в радіанах, для того щоб перевести радіани в градуси необхідно скористатись формулою: $a° = a_{rad}\frac{180°}{3.14}$

Після того як знайшли відхилення по X та Y, виводимо отримані значення на дисплей.

28. Після того як ми проаналізували всі блок-схеми та розібрались з логікою роботи окремих компонентів, можна перейти до програмування. Створіть новий проект в середовищі програмування Arduino IDE. Першим кроком необхідно додати бібліотеки які будуть використовуватись для роботи з електронними компонентами. Майже всі необхідні бібліотеки є встановленими разом з середовищем програмування. Але бібліотеку для роботи з акселерометром треба встановити окремо. Для цього в середовищі Arduino IDE перейдіть до меню «**Інструменти**» та оберіть пункт «**Керувати бібліотеками**», або натисніть сполучення клавіш «**Ctrl+Shift+I**». У вікні що відкриється введіть назву бібліотеки: ***SparkFun_MMA8452Q*** та натисніть кнопку «**Встановити**».



Після встановлення бібліотек першим кроком необхідно об’явити всі змінні та об’єкти та ініціалізувати необхідні порти.

```
#include <LiquidCrystal.h>     // Бібліотека для роботи з дисплеєм 1602
#include <math.h>              // Бібліотека для роботи з математичними функціями
#include <Wire.h>              // Бібліотека для роботи з інтерфейсом I2C
#include <SparkFun_MMA8452Q.h>// Бібліотека для роботи з акселерометром
//Підключення дисплею
LiquidCrystal lcd(12, 11, 10, 9, 8, 7);  // (RS, E, DB4, DB5, DB6, DB7)
MMA8452Q accel;                //Створення об'єкту для роботи з акселерометром
String menu[] = {"Length", "Distance", "Level"};//ініціалізація масиву, з позиціями меню
const int interruptA = 0;      //Змінна що вказує на номер переривання
volatile long pause = 100000; //Пауза в мікросекундах, для коректної роботи енкодера
volatile long lastTurn = 0;    //змінна для підрахунку часу між поворотом ручки енкодера
int trigPin = 6;               //змінні для роботи ультразвукового датчика triger і echo
int echoPin = 5;
int clk = 2;                   //змінні для роботи енкодера clk, dat і кнопка
int dat = 3;
int enterButton = 4;
int photoInt = A0;             //пін до якого підключений фотоперериврник
int button = A1;               //Окрема кнопка по якій вмикається фотоперериврник
int pos= 0;                    //початкова позиція меню
bool tempB = true;             //змінна для коректної роботи кнопки вибору в меню
//ініціалізація та налаштування всіх портів та пристроїв
void setup(){
  pinMode(clk, INPUT);
  pinMode(dat, INPUT);
  attachInterrupt(interruptA, RoteStateChanged, FALLING);
  pinMode(photoInt, INPUT);
  pinMode(enterButton, INPUT_PULLUP);
  pinMode(button, INPUT);
  lcd.begin(16, 2);
  Wire.begin();
  accel.begin();
  printMenu(pos);
  buttonP();
}
```

Після цього реалізуйте метод **loop().**

```
void loop(){
    if(buttonP()){       //якщо натиснута кнопка вибору меню
      switch(pos){       //перевіряємо чому дорівнює змінна pos
        case 0:          //якщо змінна дорівнює - 0
            lengthD();   //викликаємо метод вимірювання дожвини
            break;
        case 1:          //якщо - 1
            distance(); //метод вимірювання відстані
            break;
          case 2:        //якщо - 2
            level();     //метод вимірювання рівня
            break;
          default:       //інакше не робимо нічого
            break;
      }
    }
}
```

В методі **loop()** ми використовуємо новий оператор **switch**. Логіка роботи цього оператора схожа на логіку роботи умовного оператора **if**, але різниця полягає в тому що оператор **switch** може перевіряти змінну тільки на рівність з відповідними константами які перелічуються у вигляді операторів **case**.

Синтаксис оператора виглядає наступним чином:

(даний лістинг вказано для ознайомлення, його не потрібно переписувати в проект)

```
switch(Змінна для перевірки){
        case константа1:
            код який буде виконуватись
            коли змінна дорівнює константі1
            break;
        case константа2:
            код який буде виконуватись
            коли змінна дорівнює константі2
            break;
        case константа3:
            код який буде виконуватись
            коли змінна дорівнює константі3
            break;
        default: //робити за замовчуванням
            код буде виконуватись у випадку якщо
            змінна не дорівнює жодній константі
            break;
 }
```

В круглих дужках оператора **switch** вказується змінна яку ми хочемо порівнювати. В тілі оператора **switch** вказуються оператори **case.**

Після ключового слова **case** вказуються константи на відповідність яким буде перевірятись змінна вказана в круглих дужках. Після двокрапки пишеться код який необхідно виконати у випадку якщо змінна буде дорівнювати константі вказаній в цьому операторі **case.**

Кількість констант не обмежена, але в кінці необхідно додати оператор **default**, який буде виконуватись в тому випадку якщо змінна вказана в дужках оператора **switch** не відповідає жодній константі.

В кінці кожного оператора **case**, використовується оператор **break**. Оператор **break** дозволяє перервати виконання оператора в якому він знаходиться. Даний оператор потрібен для того щоб код не «**провалювався**» в наступні **case**. Якщо не вказати оператор **break** в кінці тіла оператора **case**, то код буде виконуватись до тих пір поки не зустрінеться оператор **break**, іншими словами буде «**провалюватись**» в інші **case** навіть якщо ті не задовільняють умову *(примітка. в деяких випадках це може бути корисно, наприклад якщо нам необхідно виконати один і той само код при кількох різних умовах).*

Далі реалізуємо методи для обробки переривань, та меню.

```
void RoteStateChanged()
{
  //якщо з попереднього спрацювання пройшло часу менше ніж "pause"
  //метод не спрацює
  if (micros() - lastTurn < pause) return;
  cli();//вимикаємо переривання
  //коли змінна dat = HIGH ручку крутять вперед
  if(digitalRead(dat)){
    lastTurn = micros();
    //якщо поточна позиція меню дорівнює останньому пункту
    //то pos=0, інакше pos+1
    pos = pos==2 ? 0 : pos+1;
    printMenu(pos);
   }else{  // коли змінна dat = LOW крутять назад
    lastTurn = micros();
    pos = pos==0 ? 2 : pos-1;
    printMenu(pos);
   }
}
```

```
void printMenu(int pos){
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("> " + menu[pos]);
    lcd.setCursor(0,1);
    if(pos + 1 > 2){
      lcd.print(menu[0]);
    }else{
      lcd.print(menu[pos+1]);
    }
    sei();      //відновлюємо роботу переривань
}
//Метод для коректного спрацювання кнопки вибору меню
bool buttonP(){
  bool r = false;
  if (!digitalRead(enterButton) == 0 && tempB == 1){
        r = true;
      }
      tempB=!digitalRead(enterButton);
      return r;
}
```

Після, реалізації можливості вибору меню, перейдемо до реалізації методів для вимірювання величин.

```
void lengthD(){
  cli();    //на час роботи методу вимикаємо всі переривання
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("Length");
  //знаходимо довжину дуги в мм знаючи діаметр
  //колеса та кількість отворів
  double arcLen = (70 *3.14)/48;
  int count = 0;
  bool temp = false;
  delay(150);
  while(!buttonP()){              //виконувати поки кнопка вибору меню не буде натиснута
    while(!digitalRead(button)){  //виконувати поки натиснута кнопка активації датчика
      int photo = digitalRead(photoInt);
      //якщо зафіксували спрацювання датчика, та перед цим спрацювання не було
      //така перевірка необхідна оскільки з огляду на швидкість роботи мікроконтролера
      //датчик може спрацювати декілька разів на один отвір
      if (photo == HIGH && temp == LOW){
        count++;
      }
      temp = photo;
      delay(20);
      double len = count*arcLen / 10;
      lcd.setCursor(0,1);
      lcd.print(String(len) + " cm     ");
    }
    count = 0;
  }
  printMenu(pos);
}
```

```
void distance(){
    cli();
    lcd.clear();
    lcd.setCursor(0,0);
    lcd.print("Distance");
    delay(1500);
    //виконувати поки кнопка вибору меню не буде натиснута
    while(!buttonP()){
        digitalWrite(trigPin, LOW);
        delayMicroseconds(5);
        digitalWrite(trigPin, HIGH);
        delayMicroseconds(10);
        digitalWrite(trigPin, LOW);
        int duration = pulseIn(echoPin, HIGH);
        int cm = (duration / 2) / 29.1;
        lcd.setCursor(0,1);
        lcd.print("Distance "+ String(cm)+"     ");
        delay(20);
      }
  printMenu(pos);
}

void level(){
   detachInterrupt(interruptA);   // вимикаємо тільки переривання 0 під час роботи методу
   lcd.clear();
   lcd.setCursor(0,0);
   lcd.print("level");
   delay(30);
   while(!buttonP()){             //виконувати поки кнопка вибору меню не буде натиснута
     if (accel.available()){      //якщо акселерометр доступний
        accel.read();             //зчитуємо дані, та ініціалізуємо змінні
        double x = accel.cz;      //з огляду на фізичне розташування акселерометра
        double y = accel.cy;
        double z = accel.cx;
        //знаходимо кути в радіанах та переводимо в градуси
        double ax = atan(x / sqrt(pow(y,2) + pow(z,2)));
        ax = ax / (3.14 / 180);
        double ay = atan(y / sqrt(pow(x, 2) + pow(z, 2)));
        ay = ay / (3.14 / 180);
        lcd.setCursor(0,1);
        lcd.print("X-> "+ String(int(ax)) + " Y -> " + String(int(ay))+"   ");
     }
     delay(100);
   }
  //вмикаємо переривання 0
  attachInterrupt(interruptA, RoteStateChanged, FALLING);
  printMenu(pos);
}
```

29. Завантажте написану програму до мікроконтролера. Після завантаження програми покрутіть ручку потенціометра поки меню не буде чітко видно на дисплеї. Крутячи ручку енкодера можна обрати необхідний пункт меню, натиснувши на ручку енкодера буде обиратись відповідний режим роботи. В режимі вимірювання довжини, датчик буде працювати тільки при натиснутій тактовій кнопці.

30. Перевірте та протестуйте всі можливості розумної лінійки, за необхідності усуньте помилки та несправності.

Самостійна робота. Змініть логіку роботи метода для вимірювання відстані **distance()**. Реалізуйте метод таким чином, щоб він робив замір відстані та виводив значення на екран тільки при натисканні тактової кнопки. Також додайте можливість перемикатись між одиницями вимірювання, наприклад між сантиметрами та дюймами, при довготривалому натисканні тактової кнопки.